change between deployments in different environments. You saw how to use property files, environment variables, configuration services, and ConfigMaps. I also showed you how to use Spring profiles to customize the application configuration based on the deployment environment. Now we need to take a step further and define a strategy to customize the entire deployment configuration depending on where we deploy an application.

In the previous section, you learned how to compose and process Kubernetes manifests together via a `Kustomization` resource. For each environment, we can specify *patches* to apply changes or additional configurations on top of those basic manifests. All the customization steps you'll see in this section will be applied without changing anything in the application source code but using the same release artifacts produced earlier. That's quite a powerful concept and one of the main features of cloud native applications.

The Kustomize approach to configuration customization is based on the concepts of *bases* and *overlays*. The k8s folder we created in the Catalog Service project can be considered a *base* : a directory with a kustomization.yml file that combines Kubernetes manifests and customizations. An *overlay* is another directory with a kustomization.yml file. What makes it special is that it defines customizations in relation to one or more bases and combines them. Starting from the same base, you can specify an overlay for each deployment environment (such as development, test, staging, and production).

As shown in figure 14.3, each `Kustomization` includes a kustomization.yml file. The one acting as the *base* composes together several Kubernetes resources like Deployments, Services, and ConfigMaps. Also, it's not aware of the overlays, so it's completely independent of them.